

Final Exam Report (DL-CV: DS-681)

REINFORCE Small Language Model Credit Assignment

Character-Level Language Modeling with Terminal Rewards

Prepared by: Shivangi Srivastava

UCID: ss5554

Source notebook: [DL_CV_Finals](#)

Item	Description
Main Algorithm	REINFORCE policy-gradient learning
Model Type	Memoryless autoregressive character-level policy
Vocabulary	<bos>, a, b, c, <eos>
Reward	1 if the generated trajectory contains the substring abc; otherwise 0
Training Steps	1500 steps without baseline and 1500 steps with running-mean baseline

Abstract

This report describes the implementation and analysis of a small character-level language model trained with the REINFORCE algorithm. The goal of the assignment was to study how to assign credit over time when the reward only appears at the end. The model starts with the beginning-of-sequence token <bos> and samples one token at a time from a learned policy until it either emits <eos> or reaches a maximum length. The reward is sparse: a trajectory gets a reward of 1 only if the substring abc appears anywhere in the generated sequence, and it receives 0 otherwise.

The implemented policy uses an embedding layer followed by a linear layer, which produces logits over the five-token vocabulary. Two training variants were used: standard REINFORCE without a baseline and REINFORCE with a running-mean baseline. Both training runs achieved a final average reward of 1.0 over 100 steps. The learned conditional distributions show that the model effectively learned the rewarded chain <bos> to a, a to b, and b to c. However, the model did not always learn to stop right after c, which fits with the reward design since extra tokens after the first appearance of abc do not lower the reward. This limitation is discussed as part of the temporal credit assignment analysis.

1. Introduction

The assignment focuses on reinforcement learning for sequence generation. Unlike supervised language modeling, where the model gets a target token at every position, this task only gives a final reward after the entire sequence has been sampled. The model must learn which earlier token choices led to success. This situation describes the temporal credit assignment problem: actions taken early in a sequence may lead to a reward that appears much later.

The specific learning goal is straightforward but valuable for analysis. The policy needs to learn how to generate the substring abc. To earn a reward of 1, the model must make several correct decisions in order: choosing a after <bos>, choosing b after a, and choosing c after b. Since the reward is seen only after the complete sequence is sampled, there is no immediate feedback to indicate that any of these intermediate actions was correct.

2. Problem Formulation

The model runs on the five tokens of its vocabulary, , a, b, c, and . The token is the only token that is used solely for the starting point and will not be part of the returned sequence of generated tokens. The model can generate and terminate its sequence on the token <eos> or can continue to generate until it has reached the length limit for generation.

Component	Specification
Vocabulary	[<bos>, a, b, c, <eos>]
Start token	<bos>
Maximum sampled length	8 tokens
Terminal condition	Stop when <eos> is sampled or max_len is reached
Reward rule	$R(\tau) = 1$ if abc appears as a substring; otherwise $R(\tau) = 0$
Learning objective	Maximize expected terminal reward through policy-gradient updates

Reward for completing a task will be represented as $R(\tau) = 1$ if "abc" has been seen in the sampled sequence; otherwise we would represent $R(\tau) = 0$. These two rewards are inherently sparse because the vast majority of random trajectories will not have seen "abc" at first. The other form of delay in these types of rewards is that the actual reward is computed after the entire trajectory is completed.

3. Model Architecture

The memoryless character network is a token-level model that uses the last seen token to predict the next. Since the transitions we want to model are local in nature, there's no need for a recurrent network or a transformer. Each token from the last time step (the previous) has its own embedding representation. From the embedding representation of the previous token, the output layer will produce logit values across the vocabulary of the next token.

Layer	Input	Output	Purpose
Embedding	Token ID in {0,1,2,3,4}	8-dimensional vector	Learned representation of the current context token
Linear	8-dimensional embedding	5 logits	Scores for the five possible next tokens
Categorical sampler	Logits	Sampled next token	Samples actions according to the policy

The forward pass returns raw logits, not probabilities. This is important because the sampling function uses `Categorical(logits=logits)`, which internally converts logits into a probability distribution.

```
class CharPolicy(nn.Module):
    def __init__(self, embed_dim: int = 8):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.output = nn.Linear(embed_dim, vocab_size)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        emb = self.embedding(x)
        logits = self.output(emb)
        return logits
```

4. Trajectory Sampling and Reward Computation

is the base case when generating the trajectories. During each iteration through the trajectory, the model generates logits for all the vocabulary tokens, from which a categorical distribution is formed and one token is sampled from that distribution. The log-probability of every sampled token is stored for later use by the REINFORCE algorithm, as it will require the log probability of each sampled token for calculating the loss. If the sample is an token, it will be stored in the final output sample sequence and its log probability will be included in the trajectory log probabilities.

The reward function will concatenate all the sampled token strings together to check for the presence of 'abc' as a substring in the complete sequence. This design allows for the model to output any sequence that contains 'abc'. It is not required that the sequence generated be 'abc'. All that is needed is that some substring in the sampled sequence contains abc.

Test Sequence	Expected Reward	Observed Reward
[a, b, c, <eos>]	1.0	1.0
[a, c, b, <eos>]	0.0	0.0
[b, a, b, c]	1.0	1.0
[a, b, <eos>]	0.0	0.0

5. REINFORCE Without Baseline

The initial training experiment utilized standard REINFORCE with no baseline. For a sampled trajectory τ , the loss was evaluated as follows:

$$L(\theta) = -R(\tau) * \sum_t \log(\pi_{\theta}(a_t | a_{-}))$$

If a sampled trajectory receives a reward of 1, then the (negative) loss result will be to increase the probability of performing each action in that trajectory, while if the sampled trajectory receives a reward of 0, then the (negative) loss will result in a zero value; therefore, the trajectory does not provide any useful, positive learning signal to the agent. Thus, there is an exploration problem due to the cold-start of many agents that due to early on in training there are very few trajectories that receive any reward; therefore, a successful trajectory is not as common early in training.

Step Range	Last 100-Step Average Reward	Interpretation
100-1100	0.000	The model had not yet discovered reliable rewarded trajectories.
1200	0.610	The model began sampling trajectories containing abc.
1300	0.990	The policy became highly successful.
1400	1.000	The model consistently sampled rewarded trajectories.
1500	1.000	Final no-baseline reward reached the maximum observed average.

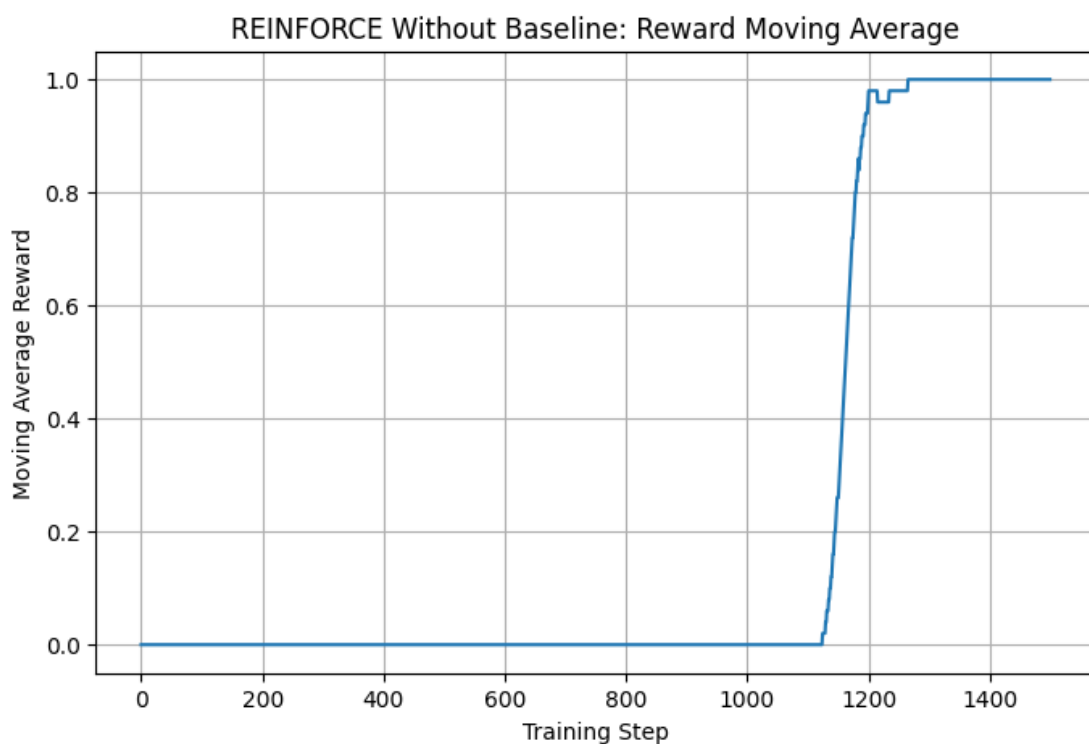


Figure 1. Moving average reward for REINFORCE without baseline.

6. REINFORCE With Running-Mean Baseline

The second experiment utilized the REINFORCE algorithm along with a running average of the rewards to train a fresh policy. The calculation of the advantage term used by REINFORCE is given as follows: advantage (A) = (R) - (b) (where " b " is an exponentially decaying moving average of the previous N rewards).

The formula for updating " b " is as follows: $b = (0.9)b + (0.1)R$. The loss function for this update is as follows: $L(\theta) = - (R(\tau) - b) * \sum(t) \log(\pi_{\theta}(a_t | a_{_}))$.

It is important to note that the introduction of a baseline does not impact the expected policy gradient since it is independent of the specific sample action; however, it acts to reduce the variance of the gradient estimator and stabilize the training. In this experiment, both models (baseline and no-baseline) ended up achieving the same final reward; however, the baseline is still necessary because it provides a more central learning signal.

Metric	Without Baseline	With Running-Mean Baseline
Training steps	1500	1500
Final 100-step average reward	1.0	1.0
Total rewards logged	1500	1500
Main learning signal	$R(\tau)$	$R(\tau) - b$
Purpose	Basic policy-gradient update	Variance reduction through advantage

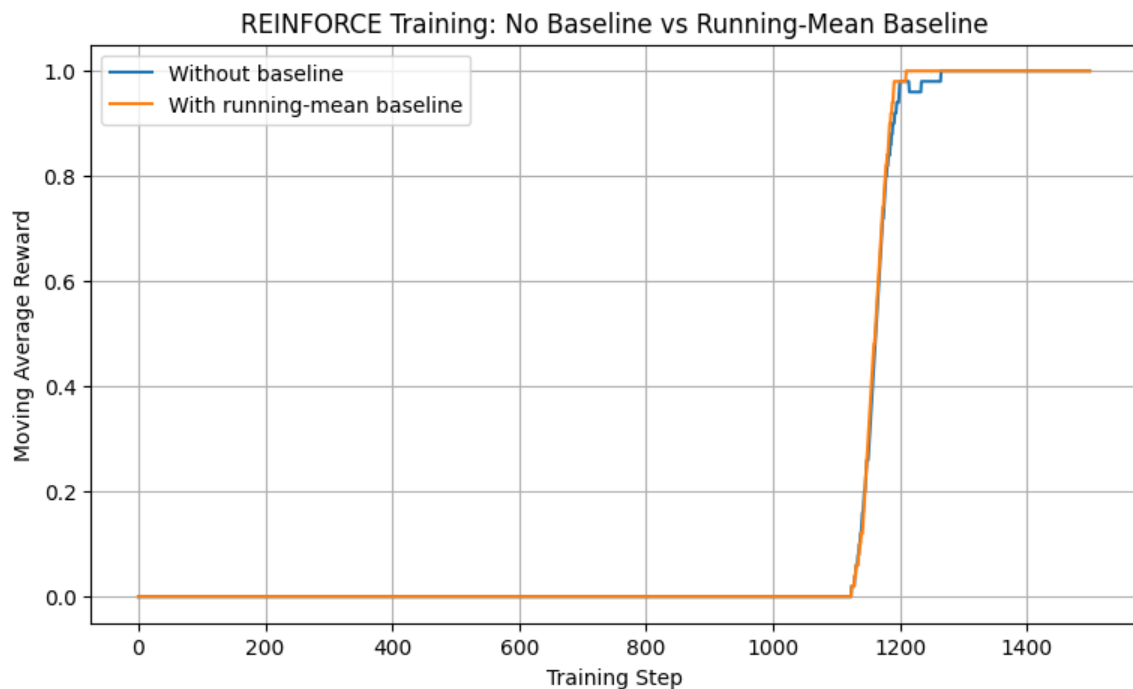


Figure 2. Moving average reward comparison: no baseline vs running-mean baseline.

7. Learned Conditional Distributions

After training, the learned baseline policy was evaluated by producing conditional probabilities of the Next Token given each of the tokens in the Evaluated contexts , a, b, and c for the next tokens and producing the conditional probabilities of the next tokens in the sequence using the learned model to determine how much probability is placed on different next token decisions.

Context	$P(<bos>)$	$P(a)$	$P(b)$	$P(c)$	$P(<eos>)$	Most likely next token
$<bos>$	0.0016	0.9918	0.0024	0.0009	0.0032	a
a	0.1018	0.0025	0.8918	0.0015	0.0025	b
b	0.0003	0.0001	0.0001	0.9992	0.0003	c
c	0.4949	0.0616	0.1802	0.1748	0.0885	$<bos>$

The distributions show the model has clearly learned the very strong reward chain of a-b-c with $P(a |) = 0.9918$, $P(b | a) = 0.8918$, and $P(c | b) = 0.9992$. The conditional probability of `<eos>` appearing after c was only 0.0885, therefore the model has behaved incorrectly based on the probability definition for an ideal ending. However, this makes sense since `<eos>` is only checked against the reward function, which only checks for the occurrence of abc in any part of the sequence. Therefore, once abc has been found, the addition of any tokens after c do not decrease the reward.

8. Sampled Trajectories From the Trained Policy

There were 10 trajectories sampled from the baseline-trained model. For each trajectory sampled, all were given a reward of 1.0 due to the fact that each trajectory had the substring of abc present. There were many sequences whose trajectories continued past the last letter of (c) of the string abc thus indicating that the reward function will not terminate immediately following c.

Sample	Generated Sequence	Reward
1	[a, <bos>, a, b, c, <bos>, a, b]	1.0
2	[a, b, c, <bos>, a, b, c, b]	1.0
3	[a, b, c, <bos>, a, b, c, <bos>]	1.0
4	[a, b, c, <bos>, a, b, c, <bos>]	1.0
5	[a, b, c, <bos>, a, b, c, c]	1.0
6	[a, b, c, a, b, c, <eos>]	1.0
7	[a, b, c, b, c, <bos>, a, a]	1.0
8	[a, b, c, b, c, c, <bos>, a]	1.0
9	[a, b, c, <bos>, a, b, c, <bos>]	1.0
10	[a, b, c, c, c, <bos>, a, <bos>]	1.0

9. Credit Assignment Analysis

The primary point of this assignment is that even at the first action there can be a gradient signal to update this action based on receiving a reward only at the end of the trajectory. REINFORCE computes the log-probability of every action using the log-probability value as a multiplier for when the reward was issued. Therefore, the first action contributes to the loss through the equation:

$$R(\tau) * (\log \pi_{\theta}(a_1 |))$$

Where τ is the trajectory, $\pi_{\theta}(a_1 | <bos>)$ represents the probability of taking action a_1 given the trajectory starts with `<bos>`, and the output of this equation is the loss associated with action a_1 . If a trajectory contains abc eventually then $R(\tau)=1$ and the model will increase the likelihood for every action in that trajectory including the first decision following the `<bos>`. If the trajectory does not contain abc then the reward $R(\tau)=0$ and therefore, the trajectory will not contribute to the model in the same manner. This represents how a delayed reward can be retroactively assigned to prior token decisions.

This effect can be seen in the high probability associated with the token 'a' preceded by `<bos>`. This means that the model learns that starting with 'a' is frequently part of a successful trajectory. From here, the model also learns

that ‘b’ generally comes after ‘a’ and that ‘c’ generally comes after ‘b’, thereby forming our base transition chain necessary to achieve reward. In addition, to some extent, the model’s less deterministic behaviour following token ‘c’ is not indicative of a failure in its learned reward objective; instead, it represents a limitation of the reward specification itself. Since the reward provided from the specification does not punish the continuation of the transition chain(s) following tokens ‘a’, ‘b’, and ‘c’, the model does not have a strong positive incentive to place the majority of its probability mass on the end of sequence token ‘’ when exposed to token ‘c’.

10. Discussion of Baseline Effects

The no-baseline and baseline experiments both reached a final 100-step average reward of 1.0 in this run. This indicates that the assignment objective was successfully solved by both variants. However, the baseline remains theoretically important. Without a baseline, the gradient estimate depends directly on sparse rewards, so many early trajectories with reward 0 provide no positive learning signal. With a running-mean baseline, the update uses the advantage $R - b$, making the gradient estimator less noisy and more stable in expectation.

In practice, the baseline is especially useful when successful trajectories are rare or when rewards have high variance. Although this particular run converged successfully in both cases, the baseline formulation is more robust and better reflects standard policy-gradient practice.

11. Limitations

Limitation	Explanation
Sparse reward	The model receives feedback only at the end of the trajectory, making early exploration difficult.
No exact stopping incentive	The reward does not require the sequence to stop after abc, so <eos> after c is not strongly learned.
Memoryless policy	The policy conditions only on the previous token, which works for this task but would not be enough for more complex dependencies.
Seed sensitivity	REINFORCE can depend on whether successful trajectories are discovered early enough during training.

12. Conclusion

In this last exam project, I performed REINFORCE on a small, autoregressive, character-level language model (CLM) using only terminal rewards and reinforcers. The implementation used a memoryless policy network, sampled trajectories with stored log-probabilities, a terminal reward function, and REINFORCE without a baseline for training.

Both versions of REINFORCE achieved a final average reward of 1.0 across 100 steps in the final reward curves.

The conditional distributions and sampled trajectories confirm that the model learned the key rewarded pattern, abc. Additionally, the project illustrates the primary temporal credit assignment concept where early

token decisions receive some learning signal since the log-probabilities associated with the tokens are multiplied by the final reward for the trajectory in the REINFORCE objective.

The most important limitation noticed in the project was that the model did not always learn to immediately stop after producing the token c since the reward function did not punish any tokens produced in excess of abc. Nevertheless, the project met the objectives of the final assignment and provided an excellent example of temporal credit assignment through delayed reinforcement learning within sequence generation.